

Lerneinheit 1 · Python-Grundlagen 1: Grundbausteine

Version 1.0

Lernziele

Nach diesem Teil können Sie ...

- im Notebook arbeiten: Zellen ausführen, Fehlermeldungen lesen, Zustand zurücksetzen,
- mit Variablen, Zahlen, Strings und Listen umgehen,
- Verzweigungen, Schleifen und eigene Funktionen schreiben,
- fertige Module mit `import` nutzen,
- erklären, warum Python-Variablen Etiketten sind und was das für Zuweisungen bedeutet,
- eine vorgelegte Lösung systematisch gegen ihre Anforderungen prüfen — mit eigenen Testfällen und begründetem Urteil.

Teil 1 von drei: Der Python-Grundkurs beginnt bei null, setzt aber Ihre C/C++-Erfahrung aus PROG1 und PROG2 voraus — er erklärt nicht, was eine Schleife ist, sondern wie Python sie schreibt und wo sie sich anders verhält. Teil 2 (Container und Daten) und Teil 3 (Struktur, Klassen, Stil) folgen mit den nächsten Lerneinheiten. Jede Übung hat eine Prü fzelle, die sofort Rückmeldung gibt, ob Ihre Lösung die vorgegebenen Testfälle besteht. Planen Sie grob drei Stunden ein, gern in mehreren Sitzungen. Starten Sie mit der Selbsteinstufung direkt unten: Sie sagt Ihnen, welche Kapitel Sie überspringen können.

Inhalt

- 1.1 [Selbsteinstufung](#)
- 1.2 [Ankommen im Notebook](#)
- 1.3 [Variablen und Zahlen](#)
- 1.4 [Strings](#)
- 1.5 [Listen](#)
- 1.6 [Verzweigungen](#)
- 1.7 [Schleifen](#)
- 1.8 [Funktionen](#)
- 1.9 [Variablen sind Etiketten](#)
- 1.10 [Module nutzen](#)
- 1.11 [Selbsttest](#)

1.1 Selbsteinstufung

Acht kleine Schnipsel. Sagen Sie für jeden die Ausgabe voraus — auf Papier, **ohne die Zelle auszuführen und ohne KI**, sonst bringt der Test nichts. Dann ausführen und vergleichen:

```

# Erst JEDE Ausgabe vorhersagen, dann ausführen und vergleichen
print("A:", 7 // 2, 7 % 2)

print("B:", f"{2 / 3:.2f}")

print("C:", [10, 20, 30][1])

t = 100
if t < 100:
    print("D: flüssig")
else:
    print("D: dampf")

words = []
for w in ["gpu", "ki"]:
    words.append(len(w))
print("E:", words)

def double(x, factor=2):
    return x * factor
print("F:", double(3), double(3, 10))

a = [1]
b = a
b.append(2)
print("G:", a)

def shout(word):
    print(word + "!")
result = shout("go")
print("H:", result)

```

So lesen Sie Ihr Ergebnis:

Falsch	Dorthin
A	1.3
B	1.4
C	1.5 — auch wenn <code>append</code> oder <code>len</code> die Hürde waren
D	1.6
E	1.7 — lag es an der Liste: 1.5
F	1.8
G	1.9 — das lohnt sich für fast alle
H	1.8 — <code>print</code> zeigt nur an, <code>return</code> gibt zurück; ohne <code>return</code> kommt <code>None</code> an

Alles richtig? Dann nehmen Sie nur noch das kurze Kapitel [1.10](#) mit und springen zum Selbsttest in [1.11](#). Einzelne Fehler? Die genannten Kapitel durcharbeiten; was der Test hier nicht abdeckt, fängt der Selbsttest am Ende mit auf.

1.2 Ankommen im Notebook

Ein Notebook besteht aus Zellen. Codezellen führen Sie mit **Umschalt+Eingabe** aus; das Ergebnis erscheint darunter. Probieren Sie es:

```
# Alles hinter einem Doppelkreuz ist ein Kommentar – für Menschen, nicht für Python
print("Hallo Tensor to GPU")
print("drei plus vier ist", 3 + 4) # print nimmt mehrere Werte, Kommas werden zu Leerzeichen
3 + 4
```

```
Hallo Tensor to GPU
drei plus vier ist 7
```

```
7
```

Drei Dinge sind hier passiert: Kommentare beginnen mit `#` und werden ignoriert (so sind auch alle Hinweise in den Kurszellen geschrieben), `print` gibt beliebig viele Werte durch Kommas getrennt aus, und der **letzte** Ausdruck einer Zelle wird zusätzlich automatisch angezeigt — deshalb sehen Sie die 7 ohne `print`.

Drei Handgriffe, die Sie vor Frust bewahren:

1. **Reihenfolge zählt, nicht Position.** Zellen wirken in der Reihenfolge, in der Sie sie ausführen. Wenn sich das Notebook seltsam verhält: Kernel neu starten (Menü *Laufzeit* bzw. *Kernel*) und die Zellen von oben bis zu Ihrer aktuellen Aufgabe erneut ausführen. *Neu starten und alle ausführen* ist die Schlusskontrolle, wenn alle Aufgaben gelöst sind — vorher scheitern die Prüfszellen der noch offenen Aufgaben erwartungsgemäß.
2. **Fehlermeldungen von unten lesen.** Die letzte Zeile des Tracebacks nennt Fehlertyp und Problem, direkt darüber steht die Zeile Ihres Codes, in der es passiert ist.
3. **Aus Versehen in einer Textzelle gelandet?** Umschalt+Eingabe stellt die Ansicht wieder her.

Und wenn es knallt? Dann lesen Sie den Traceback, finden die Zeile, korrigieren und führen erneut aus — dieser Zyklus ist das halbe Programmieren, und Sie üben ihn gleich einmal. Zwei Fehlertypen begegnen Ihnen dabei: Ein `TypeError` heißt, die Typen passen nicht zusammen (etwa eine Zahl minus einen String), ein `NameError` heißt, diesen Namen gibt es nicht — meist ein Tippfehler.

Aufgabe 1.2 #1

10 min

Der Code in der Zelle darunter soll `Endpreis: 100` ausgeben, stattdessen scheitert er gleich zweimal. Führen Sie ihn aus und arbeiten Sie sich durch die Tracebacks, einen Fehler nach dem anderen, direkt in der Zelle. Achtung: Die Zeile, die ein Traceback nennt, ist die Zeile, in der der Fehler **auffällt** — entstanden ist er manchmal weiter oben.

Zwei Vokabeln vorab: `+` verbindet zwei Strings, und `str(final)` wandelt die Zahl dafür in einen String um — beides kommt in den nächsten Kapiteln genauer.

Unter jeder Aufgabe dieses Kurses steht eine **Prüfzelle**, die Sie einfach ausführen. Sie arbeitet mit `assert`: Ein `assert` bedingung, "meldung" tut nichts, solange die Bedingung stimmt — und stoppt mit einem `AssertionError` samt Meldung, sobald sie es nicht tut. Die Meldung sagt Ihnen also, welche Erwartung Ihre Lösung verletzt. Gleich sehen Sie die erste.

```
price = 120
discount = "20"
final = price - discount
message = "Endpreis: "
print(message + str(final))
```

```
# Prüfzelle – einfach ausführen
assert final == 100, "120 minus 20 – discount muss eine Zahl sein, kein String"
assert message == "Endpreis: ", "message ist nur der Text vor der Zahl – der Tippfehler steckt in der Definitionszeile, nicht im print"
print("Alle Prüfungen bestanden ✓")
```

1.3 Variablen und Zahlen

Eine Variable entsteht durch Zuweisung — ohne Typangabe, ohne Semikolon. Der Typ hängt dabei am **Wert**, nicht an der Variablen: `type()` verrät ihn, und derselbe Name darf nacheinander Werte verschiedener Typen tragen (in C++ undenkbar). Python kennt ganze Zahlen (`int`, beliebig groß) und Fließkommazahlen (`float`) und rechnet mit den gewohnten Operatoren; `**` ist die Potenz, `//` teilt ganzzahlig, `%` liefert den Rest:

```
students = 25
groups = 4
print(students / groups)    # echte Division -> 6.25 (float)
print(students // groups)   # ganzzahlig -> 6
print(students % groups)    # Rest -> 1
print(2 ** 100)             # int wird beliebig groß – kein Überlauf
print(type(students), type(students / groups))
```

```
6.25
6
1
1267650600228229401496703205376
<class 'int'> <class 'float'>
```

Merken Sie sich den Unterschied der Divisionen: `/` liefert **immer** ein `float`, auch bei `8 / 2`. Wer ein ganzzahliges Ergebnis braucht (etwa als Index), nimmt `//` — mit zwei Feinheiten: `//` rundet nach **unten**, bei negativen Zahlen also anders als die C/C++-Ganzzahldivision (`-7 // 3` ist `-3`, nicht `-2`), und ein `int` kommt nur heraus, wenn beide Operanden `int` sind (`8.0 // 2` ergibt `4.0`).

Zwischen den Typen wandeln Sie ausdrücklich um — die Typnamen sind zugleich Funktionen:

```
print(int("42") + 1)          # String -> Zahl
print(float("0.5") * 2)       # Zahl -> float
print("Ergebnis: " + str(7)) # Zahl -> String, zum Verketteten
```

```
43
1.0
Ergebnis: 7
```

Überblick: Die eingebauten Datentypen

Python bringt eine Handvoll Grundtypen mit. Zahlen, Wahrheitswerte, Strings und Listen behandeln wir in diesem Teil; Tupel und `None` lernen Sie ebenfalls bereits kennen. Dictionaries und Sets folgen in Teil 2.

Typ	Beispiel	veränderlich?	behandelt in
<code>int</code>	42 (beliebig groß)	nein	Teil 1
<code>float</code>	0.83	nein	Teil 1
<code>bool</code>	<code>True</code> , <code>False</code>	nein	Teil 1 (Verzweigungen)
<code>str</code>	"Gehirn"	nein	Teil 1
<code>list</code>	[1, 2, 3]	ja	Teil 1
<code>tuple</code>	("Gehirn", 0.77)	nein	Teil 1 kurz, Teil 2
<code>dict</code>	{"id": 7}	ja	Teil 2
<code>set</code>	{1, 2, 3}	ja	Teil 2
<code>NoneType</code>	<code>None</code> („kein Wert“)	—	Teil 1 (Funktionen)

Was „veränderlich“ genau bedeutet und warum es wichtig ist, klärt das Kapitel [1.9](#).

Überblick: Die Operatoren

Rechnen, Vergleichen, Verknüpfen — die Operatoren, die Sie in diesem Kurs brauchen, auf einen Blick.

Gruppe	Operatoren	Anmerkung
Arithmetik	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> <code>**</code>	<code>/</code> liefert immer <code>float</code> , <code>//</code> ganzzahlig, <code>**</code> ist die Potenz
Kurzformen	<code>+=</code> <code>-=</code> <code>*=</code>	<code>x += 1</code> statt <code>x = x + 1</code> ; ein <code>++</code> gibt es nicht
Vergleiche	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	verkettbar: <code>0 <= x < 1</code>
Logik	<code>and</code> <code>or</code> <code>not</code>	ausgeschrieben statt der C-Schreibweisen <code>&&</code> und <code>Co.</code> — ab dem Kapitel 1.6
Enthalten	<code>in</code> , <code>not in</code>	für Listen, Strings und später alle Container
Identität	<code>is</code> , <code>is not</code>	„dasselbe Objekt?“ — Kapitel 1.9

Bit-Operatoren gibt es auch (`&`, `^`, `<<` und Verwandte); für diesen Kurs brauchen Sie sie nicht.

Namenskonventionen

Variablen- und Funktionsnamen schreibt Python klein, Wortgrenzen markiert der Unterstrich (snake_case): `total_seconds`, `count_above`. Durchgängige Großschreibung ist für Konstanten reserviert (`MAX_ITEMS`); Klassennamen kommen in Teil 3 dazu. Groß- und Kleinschreibung zählt: `score` und `Score` sind zwei verschiedene Namen. Umlaute wären in Python 3 sogar erlaubt, sind aber unüblich; dieser Kurs bleibt bei englischen Bezeichnungen. Der Styleguide dahinter ist [PEP 8](#).

Aufgabe 1.3 #1

10 min

Eine beliebte Aufwärmfrage in Programmier-Interviews: Zerlegen Sie `total_seconds = 9047` in `hours`, `minutes` und `seconds` (das Ergebnis ist 2 h, 30 min, 47 s). Sie brauchen nur `//` und `%`, in zwei Stufen.

```
total_seconds = 9047
# TODO: Ihre Lösung hier
...
print(hours, minutes, seconds)
```

```
# Prüfwerte – einfach ausführen
assert hours == 2, "9047 // 3600"
assert minutes == 30, "vom Rest (9047 % 3600) die vollen Minuten"
assert seconds == 47
assert hours * 3600 + minutes * 60 + seconds == total_seconds, "die Zerlegung muss sich wieder zusammensetzen lassen"
print("Alle Prüfungen bestanden ✓")
```

Zum Nachschlagen: [Python-Tutorial 3.1.1, Zahlen](#).

1.4 Strings

Text steht in Anführungszeichen (einfach oder doppelt, egal); ein eigenes `char` gibt es nicht, und Strings sind unveränderlich. Am wichtigsten für diesen Kurs sind **f-Strings**: ein `f` vor dem String, und in geschweiften Klammern dürfen Variablen und Ausdrücke stehen — mit Formatangabe hinter dem Doppelpunkt:

```

name = "Ada"
score = 0.83124
print("Hallo " + name + "!")           # + verkettet Strings
print(f"{name} erreicht {score} Punkte")
print(f"{name} erreicht {score:.2f} Punkte") # :.2f = zwei Nachkommastellen
print(f"in Prozent: {score * 100:.0f} %")  # auch Ausdrücke sind erlaubt

```

```

Hallo Ada!
Ada erreicht 0.83124 Punkte
Ada erreicht 0.83 Punkte
in Prozent: 83 %

```

Überblick: f-String-Formatangaben

Hinter dem Doppelpunkt in `{wert:...}` steht die Formatangabe. Drei Muster decken fast alles ab, was dieser Kurs braucht.

Angabe	Beispiel	Ergebnis
<code>:.2f</code>	<code>f"{0.83124:.2f}"</code>	0.83 — feste Nachkommastellen
<code>:.0f</code>	<code>f"{0.83124 * 100:.0f}"</code>	83 — gerundet, ohne Nachkommastellen
<code>:</code> , <code>,</code>	<code>f"{1234567:,.}"</code>	1,234,567 — Tausendertrennung
<code>></code> <code><</code> <code>></code> <code><</code> <code>></code> <code><</code>	<code>f"{42:>8}"</code>	42 — rechtsbündig auf Breite 8
<code><</code> <code>></code> <code><</code> <code>></code> <code><</code> <code>></code> <code><</code>	<code>f"{'ki':<8}"</code>	ki — linksbündig auf Breite 8
<code>:08.3f</code>	<code>f"{3.14159:08.3f}"</code>	0003.142 — Breite 8, führende Nullen, 3 Nachkommastellen

Die Angaben lassen sich kombinieren (Breite, dann Nachkommastellen). Alles Weitere steht in der [Format-Referenz](#) der Python-Dokumentation.

Aufgabe 1.4 #1

5 min

Legen Sie `line` an: einen f-String, der aus `title = "Gehirn"` und `score = 0.895` exakt die Zeile `Gehirn: 89.5 %` baut (eine Nachkommastelle).

```

title = "Gehirn"
score = 0.895
# TODO: Ihre Lösung hier
...
print(line)

```

```

# Prüfwelle – einfach ausführen
assert line == "Gehirn: 89.5 %", f"bekommen: {line!r}"
print("Alle Prüfungen bestanden ✓")

```

Zum Nachschlagen: [Python-Tutorial 3.1.2, Text](#) und [7.1.1, f-Strings](#).

1.5 Listen

Eine Liste sammelt Werte in eckigen Klammern — wie ein `vector`, nur ohne Typparameter. Sie hat eine Länge (`len`), man greift mit `liste[0]` auf Elemente zu (Zählung ab null!), hängt mit `append` an, entfernt mit `remove` und prüft mit `in`, ob etwas enthalten ist:

```
scores = [0.83, 0.41, 0.77]
print(len(scores))      # 3
print(scores[0])        # das erste Element: 0.83
scores.append(0.12)      # hinten anhängen
print(scores)
print(0.41 in scores)    # True
scores.remove(0.41)      # entfernt das erste Vorkommen dieses Werts
print(scores)
```

```
3
0.83
[0.83, 0.41, 0.77, 0.12]
True
[0.83, 0.77, 0.12]
```

Drei Griffe dazu, die Sie ständig brauchen: Ein **Slice** `scores[:2]` schneidet einen Ausschnitt heraus (vertieft in Teil 2), negative Indizes zählen von hinten, und die eingebauten Funktionen `max`, `min` und `sum` arbeiten direkt auf der ganzen Liste:

```
scores = [0.83, 0.77, 0.12]
print(scores[:2])        # Slice: die ersten zwei Elemente, als neue Liste
print(scores[-1])        # negative Indizes zählen von hinten: das letzte Element
print(f"{max(scores)} {min(scores)} Summe {sum(scores):.2f}")
```

```
[0.83, 0.77]
0.12
0.83 0.12 Summe 1.72
```

Eine Eigenheit, die regelmäßig Code zerstört — die **None-Falle**: Methoden wie `append` und `remove` **verändern die Liste** und geben als Ergebnis `None` zurück. Wer dieses Ergebnis trotzdem zuweist, überschreibt seine Liste: Nach `scores = scores.append(0.9)` ist `scores` keine Liste mehr, sondern `None`, und der nächste Listen-Zugriff stürzt ab. Diese Methoden werden für ihre Wirkung aufgerufen, nicht für einen Rückgabewert.

Überblick: Der Listen-Werkzeugkasten

Mit `len`, `Index`, `append`, `remove` und `in` kommen Sie durch Teil 1. Der volle Werkzeugkasten zum Nachschlagen steht hier; dabei steht `xs` für eine Liste, `x` für ein einzelnes Element und `i` für eine Position.

Werkzeug	Tut was	Gibt zurück
<code>len(xs)</code>	Anzahl der Elemente	<code>int</code>
<code>xs[i]</code> , <code>xs[-1]</code>	Element an Position <code>i</code> , letztes Element	das Element
<code>xs[1:3]</code>	Ausschnitt (Slice)	neue Liste
<code>x in xs</code>	Enthalten-Prüfung	<code>bool</code>
<code>xs.append(x)</code>	hängt hinten an	<code>None</code> — verändert <code>xs</code>
<code>xs.insert(i, x)</code>	fügt an Position <code>i</code> ein	<code>None</code> — verändert <code>xs</code>
<code>xs.remove(x)</code>	entfernt das erste Vorkommen	<code>None</code> — verändert <code>xs</code>
<code>xs.pop()</code>	entfernt das letzte Element	das Element — verändert <code>xs</code>
<code>xs.sort()</code>	sortiert an Ort und Stelle	<code>None</code> — verändert <code>xs</code>
<code>sorted(xs)</code>	sortierte Kopie	neue Liste
<code>xs.count(x)</code> , <code>xs.index(x)</code>	zählt bzw. findet <code>x</code>	<code>int</code>
<code>max(xs)</code> , <code>min(xs)</code> , <code>sum(xs)</code>	größter, kleinster Wert, Summe	eine Zahl

Die Spalte „Gibt zurück“ ist der Spickzettel gegen die `None`-Fälle von eben. `sort` gegen `sorted` und Slicing in voller Breite kommen in Teil 2.

Aufgabe 1.5 #1

5 min

Der Code in der Zelle darunter soll `[4.0, 2.5]` ergeben, bricht aber beim zweiten `append` mit einem `AttributeError` ab. Der Grund steckt eine Zeile früher: Nach dem ersten `append` ist `halved` gleich `None`, und `None` kennt kein `append`. Führen Sie die Zelle aus, erklären Sie sich beide Schritte und korrigieren Sie direkt in der Zelle:

```
halved = []
halved = halved.append(8 / 2)
halved = halved.append(5 / 2)
print(halved)
```

```
# Prüfwelle – einfach ausführen
assert halved == [4.0, 2.5], "append verändert die Liste und gibt None zurück – nicht zuweisen"
print("Alle Prüfungen bestanden ✓")
```

Zum Nachschlagen: [Python-Tutorial 3.1.3, Listen](#) und [5.1, Mehr zu Listen](#).

1.6 Verzweigungen

`if`, `elif`, `else` — der Block wird durch den Doppelpunkt eröffnet und durch die **Einrückung** abgegrenzt (vier Leerzeichen sind die Konvention; im Notebook rückt die Tab-Taste passend ein). Zwei Fehlermeldungen gehören zu dieser Syntax: Ein vergessener Doppelpunkt meldet sich als `SyntaxError`, eine Einrückung, die Python nicht lesen kann, als `IndentationError` — beide zeigen genau auf die betroffene Zeile. Vergleiche dürfen verkettet werden: `0 <= x < 1` prüft beides auf einmal:

```

score = 0.62
if score >= 0.8:
    label = "stark"
elif score >= 0.5:
    label = "brauchbar"
else:
    label = "schwach"
print(label)

if score >= 0.5 and score < 0.8:           # and, or, not verknüpfen Bedingungen
    print("brauchbar, aber nicht stark")
if 0.5 <= score < 0.8:                   # dieselbe Prüfung als verkettete Vergleiche – in Python
    print("die verkettete Fassung sagt dasselbe")

```

```

brauchbar
brauchbar, aber nicht stark
die verkettete Fassung sagt dasselbe

```

Zwei Wege, auf eine leere Liste zu prüfen: `if len(xs) > 0:` sagt es ausdrücklich, `if xs:` nutzt aus, dass leere Container als falsch zählen. Die kurze Form ist in Python üblich und gut lesbar, die lange genauso richtig — nehmen Sie, was Sie klarer finden. Ein Vorbehalt für später: Auf NumPy-Arrays mit mehreren Elementen funktioniert `if xs:` nicht; warum, klärt die NumPy-Lerneinheit.

Aufgabe 1.6 #1

5 min

Legen Sie `verdict` in Abhängigkeit von `temperature` an: unter 0 → "gefroren", 0 bis unter 100 → "flüssig", ab 100 → "dampf". Die Prü fzelle prüft das Ergebnis zur gerade eingestellten Temperatur — spielen Sie deshalb selbst mehrere Werte durch: Setzen Sie `temperature` nacheinander auf -5, 0, 99.9 und 100 und führen Sie beide Zellen jeweils erneut aus.

```

temperature = 42
# TODO: Ihre Lösung hier
...
print(verdict)

```

```

# Prü fzelle – prüft Ihr verdict zur aktuell eingestellten temperature
if temperature < 0:
    _soll = "gefroren"
elif temperature < 100:
    _soll = "flüssig"
else:
    _soll = "dampf"
assert verdict == _soll, f"bei temperature = {temperature} muss {_soll!r} herauskommen"
print(f"Prüfung für temperature = {temperature} bestanden ✓ – auch schon -5, 0, 99.9 und 100 probiert?")

```

Lernfragen 1.6 #1

1. Was gibt dieser Code aus?

```

score = 0.8
if score >= 0.8:
    label = "stark"
elif score >= 0.5:
    label = "brauchbar"
print(label)

```

Zum Nachschlagen: [Python-Tutorial 4.1, if](#).

1.7 Schleifen

Das Range-for aus C++ kennen Sie — Pythons `for` ist genau das, nur ist es hier die **einzige** Standardform: Die Schleife läuft direkt über die Elemente, ganz ohne Index. Wenn Sie Zahlen brauchen, gibt `range(n)` sie Ihnen, von 0 bis **ausschließlich** `n`; mit `range(start, stop)` geben Sie den Startwert vor — `start` gehört dazu, `stop` bleibt ausgeschlossen. Und wenn Sie beim Durchlaufen doch die Position brauchen, liefert `enumerate` je Durchlauf ein Paar aus Nummer und Element — die zwei Namen vor dem `in` übernehmen dessen Bestandteile (mehr zu Paaren im Kapitel [1.8](#)):

```
scores = [0.83, 0.41, 0.77, 0.12]

total = 0
for s in scores:           # s nimmt nacheinander jeden Wert an
    total = total + s
    print("Summe:", total)

for i in range(3):         # 0, 1, 2
    print("Durchlauf", i)

for i, s in enumerate(scores): # Position und Wert zugleich
    print(i, "->", s)
```

```
Summe: 2.13
Durchlauf 0
Durchlauf 1
Durchlauf 2
0 -> 0.83
1 -> 0.41
2 -> 0.77
3 -> 0.12
```

Daneben gibt es `while`, das läuft, solange seine Bedingung stimmt — in diesem Kurs selten, denn meist iterieren wir über Daten, deren Länge feststeht. `break` bricht jede Schleife vorzeitig ab:

```
power = 1
while power < 100:
    power = power * 2
    print(power)           # die erste Zweierpotenz über 100
```

```
128
```

Die Faustregel für die Wahl: direkt über die Elemente, solange Sie die Werte brauchen; `enumerate`, wenn zusätzlich die Position gebraucht wird; eine Index-Schleife nur, wenn Sie wirklich mit Indizes arbeiten müssen, etwa um das rechte Nachbarelement `xs[i + 1]` anzusprechen — dann heißt sie `for i in range(len(xs) - 1)`, denn das letzte Element hat keinen rechten Nachbarn mehr. Auch das ist in Python völlig in Ordnung.

Aufgabe 1.7 #1

10 min

Zählen Sie mit einer Schleife, wie viele Werte in `scores` größer als 0.5 sind, und legen Sie das Ergebnis in `strong` ab.

```
scores = [0.83, 0.41, 0.77, 0.12, 0.55]
# TODO: Ihre Lösung hier
...
print(strong)
```

```
# Prüfwelle – einfach ausführen
assert strong == 3, "0.83, 0.77 und 0.55 liegen über 0.5"
print("Alle Prüfungen bestanden ✓")
```

Aufgabe 1.7 #2

5 min

Eine Falle: Der Code in der Zelle darunter läuft ohne Fehlermeldung, zählt aber falsch. Der Interpreter kann Einrückungsfehler nur melden, wenn sie syntaktisch unmöglich sind (dann als `IndentationError`); das `pass` ist eine Anweisung, die nichts tut, ein reiner Platzhalter. Sagen Sie voraus, was `count` ergibt, führen Sie die Zelle aus und korrigieren Sie sie dann so, dass jeder Treffer gezählt wird:

```
count = 0
for s in [0.9, 0.2, 0.8]:
    if s > 0.5:
        pass
count = count + 1
print(count)
```

```
# Prüfwelle – einfach ausführen
assert count == 2, "das Hochzählen gehört IN den if-Block – Einrückung ist Logik"
print("Alle Prüfungen bestanden ✓")
```

Aufgabe 1.7 #3

10 min

Review-Aufgabe: Ein KI-Assistent hat die korrekte Lösung in der Zelle darunter geliefert, um alle Werte zu verdoppeln. Schreiben Sie sie direkt in der Zelle elementbasiert um (Ergebnis wieder in `doubled`) und überlegen Sie vorher: Gibt es in dieser Aufgabe eine Stelle, an der ein Index tatsächlich nötig ist?

```
values = [1, 2, 3, 4]
doubled = []
for i in range(len(values)):
    doubled.append(values[i] * 2)
print(doubled)
```

```
# Prüfwelle – einfach ausführen
assert doubled == [2, 4, 6, 8]
print("Alle Prüfungen bestanden ✓")
```

Lernfragen 1.7 #1

1. Wie viele Zeilen druckt diese Schleife — und welche Werte?

```
for i in range(2, 5):
    print(i)
```

Zum Nachschlagen: [Python-Tutorial 4.2. for](#) und [4.3. range](#).

1.8 Funktionen

Mit `def` verpacken Sie Logik zum Wiederverwenden. Parameter können Standardwerte haben, `return` gibt das Ergebnis zurück. Und aufgepasst: `print` zeigt nur an, `return` gibt zurück. Eine Funktion ohne `return` liefert automatisch `None`, egal wie viel sie druckt. Im Beispiel steckt noch eine kleine f-String-Neuheit: Auch die Formatangabe selbst darf Werte einsetzen — bei `digits = 1` wird aus `.{digits}f` die bekannte Angabe `.1f`:

```
def to_percent(score, digits=1):
    return f"{score * 100:.{digits}f} %"

print(to_percent(0.895))           # nutzt den Standardwert digits=1
print(to_percent(0.895, 0))        # überschreibt ihn über die Position
print(to_percent(0.895, digits=0)) # benanntes Argument: Zuordnung über den Parameternamen
```

```
89.5 %
90 %
90 %
```

Die letzte Variante heißt **benanntes Argument** und macht Aufrufe lesbar, sobald eine Funktion mehrere Parameter hat. Genau so ruft das Workbook `search(3, k=5)` auf.

Mit einem Tupel gibt eine Funktion auch mehrere Werte auf einmal zurück. Links vom Gleichheitszeichen packen Sie das Ergebnis direkt wieder aus:

```
def score_stats(scores, threshold=0.5):
    # größten Wert und die Anzahl der Werte ab der Schwelle (größer oder gleich) zurückgeben
    best = max(scores)
    hits = 0
    for s in scores:
        if s >= threshold:
            hits += 1
    return best, hits

best, hits = score_stats([0.83, 0.5, 0.77, 0.12])
print(f"bester Wert {best}, {hits} Werte ab der Schwelle") # die 0.5 zählt mit
```

```
bester Wert 0.83, 3 Werte ab der Schwelle
```

Nebenbei eine Namensfrage, wie sie in jedem Review auftaucht: Die Funktion hieß im ersten Entwurf `top_score`, aber sie liefert zwei Dinge, nicht eins. Ein Name soll die **ganze** Rückgabe ankündigen, deshalb `score_stats`.

Dasselbe Auspacken funktioniert direkt in der Schleife, wenn eine Liste aus Paaren besteht. Genau dieses Muster nutzen Sie im LE01-Lab durchgehend (mehr zu Tupeln als Datentyp in Teil 2):

```
pairs = [("San Diego", 0.83), ("Gehirn", 0.77)]
for title, score in pairs:
    print(f"{score:.2f} {title}")
```

```
0.83 San Diego
0.77 Gehirn
```

Aufgabe 1.8 #1

10 min

Schreiben Sie `count_above(values, limit)`, die zählt, wie viele Werte in `values` **echt größer** als `limit` sind: die Schleife aus dem letzten Kapitel, als Funktion verpackt. Und bevor Sie die Prü fzelle starten: Schreiben Sie in die zweite Zelle ein eigenes `assert` mit einem Testfall, der `>` von `>=` **unterscheidet**, also einer Eingabe, bei der die falsche Variante ein anderes Ergebnis liefern würde. Anforderungen in solche unterscheidenden Tests zu übersetzen ist eine Kernfähigkeit beim Review, gerade von KI-generiertem Code (eine Systematik dazu folgt am Kapitelende in der Checkliste „Wie prü fe ich eine Lösung?“). Ehrlicherweise: Ob Ihr Testfall wirklich unterscheidet, kann die Prü fzelle nicht kontrollieren; die Zelle läuft auch leer durch. Diesen Teil verantworten Sie selbst.

```
def count_above(values, limit):
    # TODO: Ihre Lösung hier
    ...
```

```
# Ihr Testfall: ein assert, bei dem >= ein anderes Ergebnis hätte als >
# TODO: Ihre Lösung hier
...
```

```
# Prüfwelle – einfach ausführen
assert count_above([0.83, 0.41, 0.77], 0.5) == 2
assert count_above([0.5, 0.5], 0.5) == 0, "echt größer – 0.5 zählt nicht"
assert count_above([], 0.5) == 0
print("Alle Prüfungen bestanden ✓")
```



Aufgabe 1.8 #2

5 min

Jetzt entwickeln Sie `count_above` in zwei kleinen Schritten weiter, statt neu anzufangen. **Schritt 1:** Ersetzen Sie oben in `count_above` testweise `>` durch `>=` — aber sagen Sie vorher voraus, welcher der bisherigen Tests dadurch scheitert. Führen Sie Funktions- und Testzelle erneut aus, prüfen Sie Ihre Vorhersage, und machen Sie die Änderung dann wieder rückgängig. **Schritt 2:** Schreiben Sie `values_from(values, limit)`, das die passenden Werte **sammelt** statt zählt — und diesmal gehört die Grenze tatsächlich dazu (größer oder gleich). Gehen Sie `count_above` dafür Zeile für Zeile durch: Was muss sich ändern, was bleibt?

```
def values_from(values, limit):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle – einfach ausführen
assert values_from([0.83, 0.41, 0.77, 0.12], 0.5) == [0.83, 0.77]
assert values_from([0.5, 0.4], 0.5) == [0.5], "die Grenze gehört diesmal dazu"
assert values_from([], 0.5) == []
print("Alle Prüfungen bestanden ✓")
```



Aufgabe 1.8 #3

5 min

Die Funktion in der Zelle darunter druckt ihr Ergebnis, statt es zurückzugeben — deshalb kommt beim Aufrufer `None` an. Korrigieren Sie sie direkt in der Zelle (eine nichtleere Liste dürfen Sie voraussetzen):

```
def mean(values):
    print(sum(values) / len(values))
```

```
# Prüfwelle – einfach ausführen
assert mean([2, 4]) == 3.0, "return statt print – der Aufrufer braucht den Wert"
print("Alle Prüfungen bestanden ✓")
```

Überblick: Wie prüfe ich eine Lösung?

Diese vier Fragen helfen Ihnen, eine Lösung systematisch gegen die Anforderungen zu prüfen — ab jetzt Ihr Standard-Check, bevor Sie eine Prü fzelle starten.

1. **Normalfall:** Liefert ein Beispiel, dessen Ergebnis Sie von Hand kennen, genau dieses Ergebnis? Ein `assert` pro Fall genügt.
2. **Grenzfall:** Was passiert am Rand — leere Liste, nur ein Element, ein Wert genau auf der Grenze?
3. **Rückweg:** Kommt das Ergebnis per `return` beim Aufrufer an, oder wird es nur gedruckt?
4. **Nebenwirkungen:** Darf oder soll die Funktion ihre Eingabe verändern — und tut sie es? Prüfen Sie die übergebene Liste nach dem Aufruf; warum eine Funktion das überhaupt kann, klärt das nächste Kapitel [1.9](#).

Die Prü fzellen dieses Kurses stellen solche Fragen für Sie — und bestätigen dabei immer nur die geprü ften Fälle. Außerhalb des Kurses stellt sie niemand; dann ist diese Liste Ihr Werkzeug, besonders beim Review von fremdem oder KI-generiertem Code.

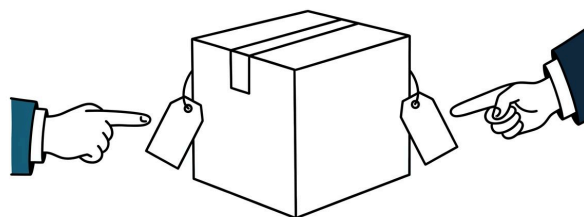
Zum Nachschlagen: [Python-Tutorial 4.8, Funktionen](#) und [4.9.1–4.9.2, Standard- und Schlüsselwortargumente](#).

1.9 Variablen sind Etiketten

Definition: Referenzsemantik

Eine Python-Variable ist ein **Name, der auf ein Objekt zeigt**, wie ein Etikett an einem Karton. Eine Zuweisung `b = a` kopiert nie das Objekt, sondern klebt nur ein zweites Etikett auf denselben Karton.

Zur Einordnung: Bei zwei eigenständigen `std::vector<int>`-Objekten kopiert die C++-Zuweisung `b = a` die **Elemente** — die Vektoren lassen sich anschließend unabhängig verändern. Bei Zeigern wird dagegen nur die **Adresse** kopiert, das Zielobjekt bleibt gemeinsam. Die C++-Referenz wiederum bindet **einmal beim Anlegen** und bleibt dann fest. Python bindet bei **jeder** Zuweisung nur den Namen neu an ein Objekt — das kommt der Zeiger-Zuweisung am nächsten, nur ohne eigene Syntax dafür. Das ist der wichtigste Denk-Unterschied zwischen beiden Sprachen und die Quelle der häufigsten Umsteiger-Fehler.



```
# Zwei Etiketten, ein Karton
a = [1, 2]
b = a          # KEINE Kopie – b zeigt auf dieselbe Liste
b.append(3)
print("a =", a)   # a sieht die Änderung mit
print("b =", b)
print("dasselbe Objekt?", a is b)
```

```
a = [1, 2, 3]
b = [1, 2, 3]
dasselbe Objekt? True
```

```
# Wer wirklich eine Kopie will, sagt es ausdrücklich
a = [1, 2]
b = list(a)     # neue Liste mit denselben Elementen
b.append(3)
print("a =", a, " b =", b, " dasselbe Objekt?", a is b)
```

```
a = [1, 2]   b = [1, 2, 3]   dasselbe Objekt? False
```

Dasselbe Prinzip entscheidet, was eine Funktion beim Aufrufer anrichtet. Sagen Sie vor dem Ausführen nicht nur voraus, was die beiden `print`-Zeilen zeigen, sondern auch **warum**: Welche der beiden Funktionen verändert den Karton — und welche klebt nur ein neues Etikett auf einen lokalen Namen?

```
def add_flag(xs):
    xs.append("!!")    # verändert das übergebene Objekt – der Aufrufer sieht es

def replace(xs):
    xs = ["neu"]       # bindet nur den LOKALEN Namen neu – der Aufrufer merkt nichts

data = [1, 2]
add_flag(data)
print("nach add_flag:", data)
replace(data)
print("nach replace: ", data)
```

```
nach add_flag: [1, 2, '!!']
nach replace:  [1, 2, '!!']
```

Und noch eine Stufe tiefer: `copy()` (oder `list(a)`) kopiert nur die **äußere** Liste. Stecken darin wieder Listen, zeigen alte und neue Liste auf dieselben inneren Kartons — eine „flache“ Kopie. Sagen Sie erst voraus, welche der beiden Listen die 99 am Ende enthält — und begründen Sie, warum:

```
nested = [[1, 2], [3]]
flat = nested.copy()    # neue äußere Liste ...
flat[0].append(99)      # ... aber dieselben inneren Listen
print("nested =", nested)
print("flat   =", flat)
```

```
nested = [[1, 2, 99], [3]]
flat   = [[1, 2, 99], [3]]
```

Der Unterschied zwischen `is` und `==`: `==` fragt „gleicher Inhalt?“, `is` fragt „dasselbe Objekt?“. Und nicht alles lässt sich überhaupt ändern: Listen sind **veränderlich** (mutable), Zahlen, Strings und Tupel sind **unveränderlich** (immutable): Bei `x = x + 1` entsteht ein neues Objekt, der alte Karton bleibt unberührt. Warum das für NumPy-Arrays und später für GPU-Speicher wichtig wird, sehen Sie im Laufe des Semesters immer wieder.

Überblick: Was passiert mit meinen Daten?

Jede Anweisung tut genau eines von dreien: einen **Namen binden**, ein vorhandenes **Objekt verändern** oder ein **neues Objekt erzeugen**. Verhält sich eine Liste unerwartet, ist die erste Frage: Welcher der drei Fälle liegt vor?

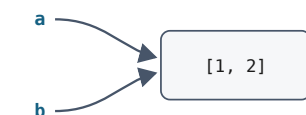
Ausgangslage für jede Zeile der Tabelle, jeweils frisch: `a = [1, 2]` und `b = a`. Es gibt dann **ein** Listenobjekt mit dem Inhalt `[1, 2]`, auf das die beiden Namen `a` und `b` zeigen.

Anweisung	Was passiert	b ist danach
<code>a.append(3)</code>	verändert das vorhandene Objekt	<code>[1, 2, 3]</code>
<code>a[0] = 7</code>	verändert das vorhandene Objekt	<code>[7, 2]</code>
<code>a = a + [3]</code>	erzeugt ein neues Objekt <code>[1, 2, 3]</code> und bindet <code>a</code> daran	<code>[1, 2]</code> , unverändert
<code>b = [5]</code>	bindet nur den Namen <code>b</code> an ein anderes Objekt	<code>[5]</code> ; <code>a</code> bleibt <code>[1, 2]</code>

Eine ausdrückliche Kopie (`list(a)`, `a.copy()`) erzeugt ein neues Objekt mit gleichem Inhalt; bei verschachtelten Listen gilt das nur für die äußere Ebene.

1 · Namen binden

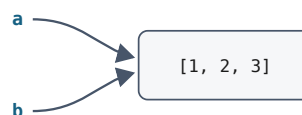
`b = a`



Zwei Namen, ein Objekt.
Nichts wird kopiert.

2 · Objekt verändern

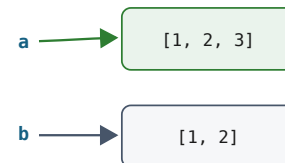
`a.append(3)`



Dasselbe Objekt, neuer Inhalt.
Auch `b` sieht die 3.

3 · Neues Objekt erzeugen

`a = a + [3]`



`a` zeigt auf das neue Objekt,
`b` weiter auf das alte.

Lernfragen 1.9 #1

1. Was gibt dieser Code aus?

```

a = [1, 2]
b = a
b.append(3)
print(len(a))
  
```

2. Welche Aussagen über `is` und `==` stimmen?

- `==` prüft, ob zwei Namen auf dasselbe Objekt zeigen
- Zwei verschiedene Listen mit gleichem Inhalt sind `==`, aber nicht `is`
- Nach `b = a` gilt immer `a is b`

3. Sie übergeben eine Liste an eine Funktion, die darin ein Element mit `append` ergänzt. Sieht der Aufrufer die Änderung?

Vertiefung: Fluent Python, Kap. 6 „Object References, Mutability, and Recycling“ — die Referenzsemantik in voller Tiefe („Variables Are Not Boxes“).

1.10 Module nutzen

Vieles, was Sie brauchen, bringt Python nicht als eingebaute Funktion mit, sondern in **Modulen**: Werkzeugsammlungen, die Sie mit `import` dazuladen. Der Punkt greift dann auf ihren Inhalt zu, und mit `as` bekommt ein Modul einen Kurznamen:

```
import math

print(math.sqrt(16))    # Funktion aus dem Modul: Modulname, Punkt, Funktionsname
print(math.pi)         # auch Werte stecken in Modulen

import numpy as np      # as vergibt den üblichen Kurznamen
print(np.sqrt(16))
```

```
4.0
3.141592653589793
4.0
```

Genau diesem Muster begegnen Sie im Workbook ab der ersten Zelle: `json` liest die Kursdaten, `numpy` (als `np`) rechnet, `time` stoppt die Zeit, `matplotlib` zeichnet, und später kommt `torch` dazu. Sie müssen sich dazu nur das Muster merken — `import`, Punkt, fertig. Woher die Module kommen und wie Sie eigene schreiben, klärt Teil 3; das Paket-Ökosystem dahinter beschreibt das Workbook im Kapitel „Python und sein Ökosystem“.



Aufgabe 1.10 #1

5 min

Ein Punkt liegt bei $x = 3$ und $y = 4$. Berechnen Sie seinen Abstand zum Nullpunkt in `length` — mit dem Satz des Pythagoras und `math.sqrt`. (Genau so eine Längenberechnung steckt übrigens im Normalize-Schritt der Kurs-Suche.)

```
import math
x = 3
y = 4
# TODO: Ihre Lösung hier
...
print(length)
```

```
# Prüfwelle – einfach ausführen
assert length == 5.0, "sqrt(3*3 + 4*4) = 5"
print("Alle Prüfungen bestanden ✓")
```

Zum Nachschlagen: [Python-Tutorial 6, Module](#) (für Teil 1 reicht der Anfang).

1.11 Selbsttest

Zum Abschluss vier Aufgaben ohne Anleitung plus zwei Review-Aufgaben. Nebenbei: Aufgaben wie diese — das längste Wort finden, ein Maximum ohne `max` bestimmen — sind Klassiker in Programmier-Interviews. Betrachten Sie den Selbsttest also ruhig als erstes kleines Interview-Training. Wenn alle Prüfwellen grün sind, sind Sie für das Lab und die nächsten Lerneinheiten gerüstet. Hakt es irgendwo, steht hinter jeder Prüfwelle, welches Kapitel hilft.



Aufgabe 1.11 #1

20 min

Selbsttest 1: `longest_word(words)` gibt das längste Wort einer Liste zurück (bei Gleichstand das erste; eine nichtleere Liste dürfen Sie voraussetzen). **Selbsttest 2:** `grade(points)` liefert "bestanden" ab 50 Punkten, sonst "nicht bestanden". **Selbsttest 3:** `report(values)` gibt einen f-String zurück wie 3 Werte, Summe 1.36 (Summe mit zwei Nachkommastellen; hier ist die leere Liste ausdrücklich zulässig). **Selbsttest 4:** `appended(values, x)` gibt eine **neue** Liste zurück, in der `x` hinten angehängt ist — die übergebene Liste bleibt unverändert.

```
def longest_word(words):
    # TODO: Ihre Lösung hier
    ...

def grade(points):
    # TODO: Ihre Lösung hier
    ...

def report(values):
    # TODO: Ihre Lösung hier
    ...

def appended(values, x):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle Selbsttest 1 – hilft: Kapitel Listen, Schleifen, Funktionen
assert longest_word(["gpu", "tensor", "ki"]) == "tensor"
assert longest_word(["ab", "cd"]) == "ab", "bei Gleichstand das erste"
print("Selbsttest 1 bestanden ✓")
```

```
# Prüfwelle Selbsttest 2 – hilft: Kapitel Verzweigungen
assert grade(50) == "bestanden", "50 reicht (größer oder gleich)"
assert grade(49.5) == "nicht bestanden"
print("Selbsttest 2 bestanden ✓")
```

```
# Prüfwelle Selbsttest 3 – hilft: Kapitel Strings, Schleifen
assert report([0.5, 0.36, 0.5]) == "3 Werte, Summe 1.36"
assert report([]) == "0 Werte, Summe 0.00"
print("Selbsttest 3 bestanden ✓")
```

```
# Prüfwelle Selbsttest 4 – hilft: Kapitel Variablen sind Etiketten
_xs = [1, 2]
_neu = appended(_xs, 3)
assert _neu == [1, 2, 3]
assert _xs == [1, 2], "die übergebene Liste darf sich nicht verändern – erst kopieren, dann anhängen"
assert _neu is not _xs, "es muss eine NEUE Liste sein, kein zweites Etikett"
print("Selbsttest 4 bestanden ✓")
```

Aufgabe 1.11 #2

15 min

Selbsttest 5, Review: Ein KI-Assistent liefert die Fassung von `best_pair(pairs)` in der Zelle darunter — sie soll aus einer nichtleeren Liste von `(titel, score)`-Paaren das Paar mit dem höchsten Score zurückgeben, bei Gleichstand das erste. Sie enthält zwei Fehler. Gehen Sie vor wie bei echtem Review: Erst direkt in der Zelle korrigieren, dann **einen eigenen Mini-Test schreiben** (ein `assert` mit einem Fall, den Sie für aussagekräftig halten — zum Beispiel lauter negative Scores), und erst danach die Prüfwelle laufen lassen. Und behalten Sie im Kopf: Bestandene Prüfwellen bestätigen nur die geprüften Fälle, der Abgleich mit der Aufgabenstellung bleibt Ihre Arbeit.

```
def best_pair(pairs):
    best = pairs[0]
    for title, score in pairs:
        if score >= best[1]:
            best = (title, score)
    print(best)
```

```
# Prüfwelle Selbsttest 5 – hilft: Kapitel Funktionen (print/return), Verzweigungen
assert best_pair([("A", 0.5), ("B", 0.9)]) == ("B", 0.9), "es muss ein Wert zurückkommen, nicht None"
assert best_pair([("A", 0.7), ("B", 0.7)]) == ("A", 0.7), "bei Gleichstand das erste Paar"
print("Selbsttest 5 bestanden ✓")
```

Aufgabe 1.11 #3

10 min

Selbsttest 6, Review mit offenem Ausgang: Noch eine KI-Fassung — diesmal sagt Ihnen niemand, ob sie Fehler enthält. `clip(values, limit)` soll eine **neue** Liste zurückgeben, in der jeder Wert über `limit` durch `limit` ersetzt ist; die übergebene Liste bleibt unverändert. Die Fassung steht ausführbar in der nächsten Zelle. Entscheiden Sie wie im echten Review: Erfüllt sie die Anforderungen? Prüfen Sie mit der Checkliste „Wie prüfe ich eine Lösung?“ aus dem Kapitel [1.8](#), korrigieren Sie nur, was wirklich falsch ist — und machen Sie Ihre Prüfung sichtbar: **Notieren Sie Ihr Urteil in einem Kommentar und ergänzen Sie zwei eigene Tests mit kurzer Begründung.** Führen Sie die Zelle auf jeden Fall aus, auch wenn Sie nichts korrigieren.

```
# Die zu prüfende KI-Fassung – korrigieren Sie nur, was wirklich falsch ist
def clip(values, limit):
    result = []
    for v in values:
        if v > limit:
            result.append(limit)
        else:
            result.append(v)
    return result

# Ihr Urteil als Kommentar, dazu zwei eigene Tests mit kurzer Begründung:
# TODO: Ihre Lösung hier
...
```

```
# Prüfwelle Selbsttest 6 – hilft: Kapitel Funktionen, Variablen sind Etiketten
_xs = [0.9, 0.5, 0.1]
assert clip(_xs, 0.5) == [0.5, 0.5, 0.1], "über der Grenze kappen, die Grenze selbst bleibt"
assert _xs == [0.9, 0.5, 0.1], "die übergebene Liste bleibt unverändert"
_res = clip(_xs, 1.0)
assert _res == _xs and _res is not _xs, "auch wenn nichts zu kappen ist: eine neue Liste, nicht die übergebene"
print("Selbsttest 6 bestanden ✓")
```

Außerdem gibt es in Python: das `match`-Statement, den Walrus-Operator `:=`, Bit-Operatoren und `input()` für Tastatur-Eingaben (in Notebooks unüblich — hier ändern Sie stattdessen Variablen direkt in den Zellen). Für diesen Kurs brauchen Sie das alles nicht; Material dazu sammelt später der optionale Teil „Advanced Python“.

Nächste Schritte

- Alle Prüfwellen grün? Dann weiter zum Lab im Workbook (`LE01_workbook.ipynb`), dort setzen Sie diese Bausteine an der echten Suche ein.
- Teil 2 des Grundkurses (Container und Daten) kommt mit der nächsten Lerneinheit.
- Zum Nachschlagen neben dem Kurs: das [offizielle Python-Tutorial](#), Kapitel 3 bis 5.

Quellen und Referenzen

- Offizielles Python-Tutorial (docs.python.org), Kap. 3–5 — deckungsgleiche Begleitlektüre; die Kapitel dieses Teils nennen ihre Abschnitte direkt.
- L. Ramalho: *Fluent Python*, 2. Aufl., Kap. 6 — Vertiefung zur Referenzsemantik.
- Software Carpentry: *Plotting and Programming in Python* (CC BY 4.0) — kleinschrittige freie Alternative für den Einstieg.